

Uniswap v4 Periphery PR #585 Review



August 12, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	5
Privileged Roles	6
Low Severity	7
L-01 Force-Exit Does Not Allow Custom Removal Bounds Or Hook Data	7
Notes & Additional Information	8
N-01 Inconsistent Handling Of Permissions Adapter Currencies Between Position Manager And Router	8
Client Reported	9
CR-01 Renouncing Adapter Ownership Permanently Disables the Force-Exit	9
CR-02 Subscribers Can Indefinitely Prevent the Force-Exit of a Position	10
Conclusion	11
Appendix	13
Issue Classification	13

Summary

Type	DeFi	Total Issues	4 (4 resolved)
Timeline	From 2026-07-31 To 2026-08-10	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (1 resolved)
		Notes & Additional Information	1 (1 resolved)
		Client Reported Issues	2 (2 resolved)

Scope

OpenZeppelin performed a review of the `Uniswap/v4-periphery` repository at the `407731f4` commit, which remediates the reported issue by moving the hook allowlist to `PermissionsAdapter` and enforcing it across the router and the position manager.

In scope were the following files:

```
v4-periphery/src
├── V4Router.sol
├── base
│   └── DeltaResolver.sol
├── hooks
│   └── permissionedPools
│       ├── BaseAllowlistChecker.sol
│       ├── PermissionedPositionManager.sol
│       ├── PermissionedV4Router.sol
│       ├── PermissionsAdapter.sol
│       ├── PermissionsAdapterFactory.sol
│       ├── interfaces
│       │   ├── IAllowlistChecker.sol
│       │   ├── IPermissionsAdapter.sol
│       │   └── IPermissionsAdapterFactory.sol
│       └── libraries
│           └── PermissionFlags.sol
└── universal-router/contracts
    ├── base
    │   └── Dispatcher.sol
    ├── modules
    │   ├── Payments.sol
    │   └── uniswap
    │       └── v4
    │           └── V4SwapRouter.sol
```

The `universal-router` files were reviewed from source, since that repository builds separately. The deployed `PermissionedHooks` contract falls outside both repositories and was reviewed through the mirror in `test/hooks/permissionedPools/mocks/MockPermissionedHooks.sol`.

System Overview

A permissioned token is exposed to Uniswap v4 through a `PermissionsAdapter`, an ERC-20 wrapper whose only permitted holder is the `PoolManager`. Adapter balances enter the `PoolManager` exclusively through `wrapToPoolManager`, restricted to addresses in `allowedWrappers`, and leave when the `PoolManager` transfers the adapter to a recipient, at which point `_update` burns it and releases the underlying token. Trading and liquidity rights are expressed as the `SWAP_ALLOWED` and `LIQUIDITY_ALLOWED` flags, resolved through an issuer-supplied `IAllowlistChecker`, alongside a global `swappingEnabled` pause.

Enforcement is layered. The pool's hook validates each swap and liquidity modification, and the official periphery contracts, `PermissionedV4Router` and `PermissionedPositionManager`, repeat those checks in their `_pay`, `_take`, `_mint` and `_increase` paths.

The reported issue was that a verified adapter was bound to no particular pool or hook. Anyone could initialize a pool pairing the adapter with another token and no hook, where `beforeSwap` never ran and the router's checks, which lived only in `_pay`, did not observe an adapter received as swap output. `_update` then released the underlying token without consulting the flags or the pause. The reviewed commit adds an adapter-owned `allowedHooks` allowlist, consulted by `_validatePoolKey` on every swap hop and by `_checkAllowedHooks` on every mint and liquidity increase, and overrides `_take` in the router to check `swappingEnabled` and `SWAP_ALLOWED`. Since creating an adapter balance requires `wrapToPoolManager` and the official wrappers now refuse pools whose hook is not allow-listed, such a pool holds no adapter balance for anyone to take.

Security Model and Trust Assumptions

- Entries in `allowedHooks` are trusted to enforce the permissions. Nothing on-chain verifies that an allow-listed hook implements the relevant callbacks, or that it performs any check when it does.

- Entries in `allowedWrappers` are trusted to report `msgSender` honestly, and to expose no operation that would convert an adapter balance into a freely transferable ERC-6909 claim.
- The underlying permissioned token is trusted to enforce its own holder allowlist on transfer. It remains the final gate on delivery in every path described in this report.
- `PoolManager.unlock`, `PoolManager.swap` and pool initialization are permissionless in v4-core. Any contract may interact with a pool that holds an adapter balance, and the hook is the only enforcement point common to every caller.

Privileged Roles

- **Adapter owner.** Set at creation by `PermissionsAdapterFactory` and transferable through `Ownable2Step`. Controls `allowedWrappers`, `allowedHooks`, `swappingEnabled` and the allowlist checker, and is the only account that may call `unwindPosition` for positions in pools containing its adapter. Fully trusted.
- **Allowlist checker.** Supplied by the adapter owner and replaceable at any time. Determines the `SWAP_ALLOWED` and `LIQUIDITY_ALLOWED` flags for every account.

Low Severity

L-01 Force-Exit Does Not Allow Custom Removal Bounds Or Hook Data

The `unwindPosition` function builds the parameters for its `BURN_POSITION` action with the minimum amounts and the hook data `hardcoded` to `uint128(0)`, `uint128(0)` and `bytes("")`. All three values are passed through to the inherited `PositionManager._burn`, where the bounds are checked by `validateMinOut` and the hook data is forwarded to `poolManager.modifyLiquidity`, reaching the pool's remove-liquidity callbacks. The admin performing a force-exit therefore has no control over either.

Consequently, a pool whose hook requires data on removal rejects the force-exit while the liquidity provider's own `BURN_POSITION` still succeeds, since that path accepts data from the caller, and the burn always executes at whatever price the pool holds, so it can be sandwiched. Only a custom allow-listed hook is affected, as the canonical `PermissionedHooks` declares no remove-liquidity callbacks, but the contracts are immutable and this cannot be corrected without redeploying the position manager.

Consider exposing the minimum amounts and the hook data as parameters of `unwindPosition`, so that the admin can supply values appropriate to the pool. Passing zeros and empty bytes preserves the current behavior exactly, so existing expectations are unaffected where the defaults are adequate.

Update: Resolved in [pull request #586](#).

Notes & Additional Information

N-01 Inconsistent Handling Of Permissions Adapter Currencies Between Position Manager And Router

The `PermissionedPositionManager` contract overrides the `_sweep` function so that, when the swept currency is a verified permissions adapter, the underlying permissioned token is transferred instead of the adapter itself. This override is required because of the way `_take` interacts with `PermissionsAdapter._update`. When an adapter is taken to the caller's own address, the adapter is burned and the underlying permissioned token is transferred in its place, so a contract that takes an adapter to itself never holds the adapter, only the underlying token.

The router does not implement an equivalent override. Sweeps in `UniversalRouter` are resolved through the `Payments` library, which reads the balance of the currency it is given. Since only the `PoolManager` may hold an adapter balance, a `SWEEP` action naming an adapter currency always observes a balance of zero. When `amountMin` is zero, the call does not revert, and the underlying token is left in the router while the action plan appears to have executed successfully. Those funds then remain claimable by any subsequent sweep of the underlying token. Similarly, the `TRANSFER` and `PAY_PORTION` actions always revert for adapter currencies, since both call `transfer` on the adapter and `_update` rejects every sender other than the `PoolManager`.

As a result, the same action is handled correctly in one contract and silently incorrectly in the other, which may cause integrators that route adapter output to the router to lose funds.

Note that this behavior was determined by reviewing the `UniversalRouter` and `PermissionsAdapter` implementations and has not been reproduced in this repository, since `UniversalRouter` is not part of it.

Consider resolving verified adapter currencies to their underlying permissioned token in the router's `Dispatcher`, matching the resolution already applied in `PermissionedPositionManager`, so that the same action behaves identically in both contracts. Note that `Payments.sweep` transfers the entire balance held by the router to the specified recipient, so the amount attributable to the current caller should be taken into

account. Alternatively, consider rejecting `ADDRESS_THIS` as a take recipient for verified adapter currencies, so that the underlying token is never left in the router at all.

Update: Resolved in [pull request #499](#).

Client Reported

CR-01 Renouncing Adapter Ownership Permanently Disables the Force-Exit

The `PermissionsAdapter` contract extends `Ownable2Step`, which overrides `transferOwnership` and `_transferOwnership` so that a handover only completes when the incoming owner calls `acceptOwnership`. Passing `address(0)` to `transferOwnership` therefore has no lasting effect. However, `Ownable2Step` does not override `renounceOwnership`, so the inherited `Ownable` implementation applies and sets the owner to `address(0)` in a single call, with no handshake and no way to reverse it.

`PermissionedPositionManager._getOwner` returns `IPermissionsAdapter.owner()` for a verified adapter, and `unwindPosition` requires the caller to match either `admin0` or `admin1`. Once the owner is zero, both resolve to `address(0)`, which no caller can satisfy, so the force-exit becomes permanently unavailable for every position in every pool that uses that adapter.

The same condition also changes how failed unwinds are resolved. The `_unwindWithFallback` function branches on `admin == address(0)` and mints an ERC-6909 claim to the liquidity provider. Claim transfers in `ERC6909Claims` do not pass through `PermissionsAdapter._update`, so the result is a freely transferable instrument denominated in the restricted asset, held by the party the unwind was intended to remove. With a live owner, the currency is delivered to the admin instead, or minted as a claim to the admin, and the liquidity provider never receives one.

Consider overriding `renounceOwnership` so that it reverts, which makes `_getOwner` non-zero by construction for a verified adapter and removes both consequences at once. Note that this encodes a product decision: adapter ownership becomes transferable but not disposable. That is consistent with the existing choice of `Ownable2Step`, but it is a policy change and is worth confirming explicitly.

Update: Resolved in [pull request #587](#).

CR-02 Subscribers Can Indefinitely Prevent the Force-Exit of a Position

The `unwindPosition` function allows a permissions adapter admin to force-exit a liquidity provider once its permissions have been revoked. It grants itself approval over the position by writing `getApproved[tokenId] = msg.sender`, and then executes the `UNSUBSCRIBE`, `BURN_POSITION` and two `UNWIND_WITH_FALLBACK` actions inside a single `poolManager.unlock` call.

The `UNSUBSCRIBE` action invokes `notifyUnsubscribe` on the position's subscriber. The `approve` function in `ERC721Permit_v4` is not restricted while the `PoolManager` is unlocked, and requires only that the caller is the token owner or an approved operator. A subscriber that is also the position owner, or an operator for it, can therefore overwrite the approval during that callback. The `BURN_POSITION` action that follows then fails its `onlyIfApproved` check, and the entire unlock reverts, which also rolls back the unsubscribe. The subscriber remains attached and the same sequence can be repeated on every subsequent attempt.

No alternative route is available to the admin, since `Notifier.unsubscribe` is itself `onlyIfApproved` and `transferFrom` reverts with `TransferDisabled` on this position manager.

As a result, a liquidity provider that sets a hostile subscriber on its own position can block the compliance force-exit indefinitely, which defeats the recovery mechanism the function exists to provide.

Consider performing the unsubscribe in a separate unlock and re-establishing the approval after that unlock has closed, so that no subscriber callback can run between the approval and the burn. Note that reordering the actions within a single unlock is not sufficient, because executing `BURN_POSITION` first reaches `_removeSubscriberAndNotifyBurn`, which is neither gas-capped nor wrapped in `try/catch` and can therefore be reverted directly by the subscriber.

Update: Resolved in [pull request #586](#).

Conclusion

The audited changes bind a permissions adapter to the set of hooks its owner has approved, enforcing that allowlist in the router on every swap hop and in the position manager on every mint and liquidity increase. They also extend the router's permission checks to adapters received as swap output, which the previous implementation only applied to adapters settled as swap input.

No high or critical severity issues were identified, and the remediation closes the reported bypass. Creating an adapter balance requires `wrapToPoolManager`, only an `allowedWrappers` address may call it, and both contracts now refuse pools whose hook has not been approved, so such a pool holds no adapter balance and no route was found that places one there.

The codebase is well structured and thoroughly tested, and each fix produced during the engagement was accompanied by regression tests that fail against the parent commit, which made the effect of every change straightforward to verify.

Two themes are worth carrying forward. First, the hook is a required part of the permission system rather than a redundant layer over the periphery. The checks in `PermissionedV4Router` and `PermissionedPositionManager` apply only to callers that use those contracts, while `PoolManager.unlock` and `PoolManager.swap` are permissionless, so a hook's `beforeSwap` and `beforeAddLiquidity` are the only checks common to every caller. `PermissionedHooks` implements them, and pools using it are enforced in both directions: `beforeSwap` rejects swaps submitted through any contract that is not an approved wrapper, and `beforeAddLiquidity` rejects liquidity added through the plain `PositionManager`. Since `allowedHooks` records a boolean with no requirement on what the hook does, approving a hook with insufficient checks is a configuration error that removes enforcement for every caller outside the periphery. Documenting the checks an approved hook must implement, and rejecting hooks that cannot implement them, would make that harder to get wrong.

Second, an adapter is not an ordinary ERC-20 from the perspective of a contract that holds balances, since taking one to a contract's own address delivers the underlying token instead. Helpers that reason about balances need to account for that distinction, and N-01 arose where one did not.

The Uniswap team was very responsive throughout the review, providing prompt clarifications on the intended trust model and delivering fixes for the reported issues within the engagement. We would like to thank them for their collaboration and for making themselves available to answer our questions.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.